PROJECT KINETICA — MASTER STUDY GUIDE

Closed-Loop Perception-Driven Intelligent Traffic Controller & Emergency Green Wave Preemption

Course: BCSE497J Project I (Fall 2026-27)	**Faculty Guide:** Dr. R. Krithiga (SCOPE, VIT Chennai)
Team: 23BDS1148 (Vaibhav. P. K) & 23BDS1168 (Mithil Girish)	**Target Evaluation:** Project Review, Technical Viva & Panel Demo

1. Executive Summary: What Problem Does Kinetica Solve?

Modern urban intersections in Indian cities (such as Chennai's OMR IT Expressway) rely predominantly on **fixed-timer signal controllers**. These controllers cycle through green lights on rigid schedules (e.g., exactly 45 seconds per approach) regardless of whether one road has a 100-meter gridlock or is completely empty. When ambulances or emergency vehicles encounter these intersections, they are forced to brake, honk, and wait behind civilian traffic queues, frequently risking lethal t-bone collisions when forcing their way through blind red lights.

Project Kinetica replaces this open-loop, rigid infrastructure with an end-to-end **Closed-Loop Cyber-Physical System (CPS)**. It integrates four coordinated engineering modules:
• **Vision Pipeline:** Edge cameras run YOLOv8 object detection combined with MoRTH AIS-125 livery decomposition to compute true queue lengths in meters and detect emergency vehicles with zero false alarms.
• **Actuation Engine:** Demand-responsive signal controller that models traffic arrivals as a Poisson stochastic process and dynamically calculates optimal green times using Webster's minimum-delay formula.
• **Priority Preemption:** An O(log N) urgency Max-Heap that immediately overrides signal cycles for ambulances and coordinates with a NetworkX directed city graph to pre-clear downstream traffic signals into an uninterrupted *Green Wave Corridor*.
• **Predictive Analytics:** A CART decision-tree regressor that forecasts post-preemption bottlenecks, supported by formal non-parametric hypothesis testing proving statistically significant wait-time reductions.

2. Deep Dive: What Every Page in the Application Does

When presenting or demonstrating the software to your teacher or evaluation panel, navigate systematically through these five views:

Page 1: Network Overview View (Chennai Arterial GIS Map)

• **Purpose:** Central command dashboard for municipal traffic operators, visualizing arterial traffic, connected intersections, and hospital emergency corridors.
• **Core Features:**
- *Surveyed Coordinate Mapping:* Plots real Chennai junctions (Madhya Kailash IX-101, TIDEL Park IX-102, SRP Tools IX-103, Sholinganallur IX-104) and Level-1 Trauma Centers (Apollo OMR, Gleneagles Global Health City).
- *Ambulance Dispatch Trigger [DISPATCH AMBULANCE & CLEAR ALL GREEN]:* Clicking the dispatch button initiates emergency preemption along the corridor to the selected hospital.
- *Dynamic SVG Ambulance Simulation:* An AIS-125 top-down vector ambulance model travels along the planned route. Features forward headlights, dual alternating cyan/red roof strobes, and shortest-path angular bearing smoothing so it never flips upside down.
- *Independent Stationary Speed HUD:* An upright pill floating above the vehicle displays live speed (e.g., 42 KM/H) without rotating with the vehicle.
- *CCTV vs. Simulation Toggle:* Switches between synthetic approach traffic and live recorded video playback for junction IX-104.
- *Elevated Floating Controls [ZOOM IN (+)], [ZOOM OUT (-)], [RECENTER TARGET]:* Positioned at bottom-right with safe clearance above the bottom telemetry bar, preventing viewport overlap or clipping.
- *Bottom Telemetry Strip:* Real-time display of total route distance (km), estimated travel time, minutes saved, and corridor preemption status (CLEARED_GREEN_WAVE).

Page 2: Intersection Detail View (CAD 4-Way Junction Simulator)

• **Purpose:** Microscopic simulation of a physical 4-way intersection (Sholinganallur IX-104) under Indian roadway geometry.
• **Core Features:**
- *Strict Indian Left-Hand Traffic (LHT):* Road carriageways, dividers, stop lines, and pedestrian crossings follow IRC standards.
- *IRC:SP:41 Channelized Free-Left Slip Lanes:* Four dedicated triangular traffic islands at the corners allow continuous bypass left turns without waiting for signal green lights.
- *CAD Vector Fleet:* Distinct top-down renderers for Indian vehicle classes: Bajaj RE auto-rickshaws (yellow/green roof), Chennai MTC transit buses, motorcycles, cars, and ambulances.
- *Signal Timing Donut Rings:* Visual countdown timers displaying active phase time remaining (green/yellow).
- *Manual Approach Overrides:* Allows operators to manually force green or red on any approach for testing.

Page 3: Vision Monitor View (Multi-Camera AI Perception)

• **Purpose:** Live CCTV surveillance and computer vision inference console showing what the traffic cameras see and how the AI processes the video stream.
• **Core Features:**
- *Quad Camera Grid:* Four simultaneous camera streams representing North (OMR Inbound), South (OMR Outbound), East (ECR / EMS Link), and West (Medavakkam Rd).
- *Live Bounding Box Overlays:* Renders color-coded bounding boxes, confidence scores, and class labels (Green for civilian vehicles, Red for Emergency).
- *Hardware Telemetry Badge:* Shows live inference latency (e.g. 18.5 ms), frame rate (54.0 FPS), and frame resolution (1280×720).
- *Livery Decomposition & Verification:* Demonstrates the MoRTH AIS-125 classifier rejecting red passenger cars and correctly confirming ambulances with flashing beacons.
- *Multi-Frame Hysteresis:* Requires emergency presence across 2-3 consecutive frames before triggering preemption, eliminating optical glare false alarms.

Page 4: Analytics & Statistical Validation View

• **Purpose:** Proves scientifically that Kinetica outperforms fixed-timer controllers and provides predictive queue intelligence.
• **Core Features:**
- *Decision Tree Bottleneck Model:* Visualizes the trained CART regressor predicting secondary queue surges after emergency preemption.
- *Feature Importance Chart:* Demonstrates that measured queue length accounts for 59.7% of downstream delay, proving visual queue estimation is essential.
- *Formal Hypothesis Test Card:* Displays the Mann-Whitney U test results ($p = 0.0000$, H_0 rejected) and effect size (16.07s wait reduction).
- *Poisson Goodness-of-Fit Card:* Displays Chi-square test verifying traffic inter-arrival distributions.
- *Cumulative Distribution Function (CDF):* Graphical comparison showing Kinetica's wait-time curve shifted significantly leftward compared to fixed timers.

Page 5: Settings & Controller Configuration View

• **Purpose:** Configuration panel for municipal traffic engineers and hardware technicians.
• **Core Features:**
- *Actuation Algorithm Switcher:* Toggle between Proportional Volume Allocation, Gap Extension, and Phase Skipping.
- *Hardware Relay Configuration:* Map signal outputs to NVIDIA Jetson GPIO pins, Modbus IP Relays, or Mock relay controllers.
- *Camera Stream Management:* Configure RTSP/HTTP endpoints and assign cameras to intersection approaches.

3. Technical Model Inventory: Detailed Specification of All Models Used

This section provides the comprehensive mathematical, algorithmic, and operational breakdown of every model utilized across the system, categorized by functional subsystem:

3.1 Path Creation & Corridor Routing Models

• **1. TomTom Routing Engine (Base Network Graph):** A* graph search over physical road topology. Inputs: GPS coordinates, vehicle speed profiles, and real-time probe slowdowns. Outputs: Uncongested base travel time, traffic delay seconds, and road coordinate polylines.
• **2. Kinetica Camera-Weighted Graph Optimization Engine (route.ts):** Intercepts TomTom candidate paths and dynamically scores them using live edge CCTV queue telemetry. Evaluates up to 3 candidate paths to select the route with minimal combined road and signal queue delays.
• **3. NetworkX Directed Multigraph Corridor Lookahead Router (graph_router.py):** Directed graph G=(V,E) where vertices V are intersections and edge weights are travel times. A greedy lookahead algorithm projects emergency trajectories 3 to 5 hops ahead, issuing preemption calls to downstream signals before the ambulance arrives.
• **4. Quadratic Bezier Curve Waypoint Interpolator:** Parametric curve B(t) = (1-t)²P0 + 2(1-t)tPmid + t²P1 (t ∈ [0,1]) generating smooth road geometry waypoints when external internet routing APIs are unreachable.

3.2 Signal Timing Calculation & Actuation Models

• **1. Modified Webster-Kinematic Green Split Model (engine.py):** Dynamically scales green phase duration as Δg = clamp((2.0 × Q) / S, 0.0s, 60.0s) where S = 1900 veh/hr/lane is the saturation flow rate. Total green is g = gmin + Δg.
• **2. Stochastic Poisson Arrival & EWMA Rate Tracking Model (arrival_model.py):** Models arrivals as counting process P(N(t)=k) = (λt)^k e^(-λt)/k! with inter-arrivals following Exponential(λ). The arrival rate λ is continuously tracked using an Exponentially Weighted Moving Average (α=0.20, window=60s) and validated via Chi-Square goodness-of-fit.
• **3. Proportional Volume Allocation Model (strategies/proportional.py):** Allocates cycle time C=100s across approaches proportionally: gi = clamp(C × [vi / Σv], gmin, gmax).
• **4. Density Gap-Out & Headway Extension Model (strategies/gap_extension.py):** Evaluates inter-vehicle headways in real time. If headway exceeds τgap = 2.5s or density drops below 0.02 veh/m, cuts green early and reclaims wasted green for queued approaches.
• **5. Circular Phase-Skipping Coordinator (strategies/phase_skipping.py):** Bypasses approaches with zero demand (vi = 0), saving 15 to 30 seconds per empty phase.
• **6. Urgency Binary Max-Heap Ranking Model (heap.py):** Ranks approaches using Urgency = M × [1 + 20×Density] + 1.0×WaitTime. Priority multiplier M=1000× for ambulances guarantees O(log N) promotion to heap root in under 1 microsecond.
• **7. Emergency Clearance State Machine (emergency_hold.py):** Enforces fail-safe transitions: IDLE → YELLOW CLEARANCE (3.0s) → GREEN HOLD (28.0s) → CLEARED, with a 10s minimum dwell lock preventing signal flapping.

3.3 Perception, Vehicle Detection & Livery Classification Models

• **1. YOLOv8 Edge Nano Multi-Class Object Detector (best.onnx):** Single-stage convolutional neural network with 3.2M parameters, anchor-free decoupled split head, and C2f cross-stage partial modules. Runs at 18.5 ms per frame (54 FPS) on CPU, detecting 8 vehicle classes.
• **2. MoRTH AIS-125 Livery Decomposition & Flashing Strobe Model (classify.py):** Structural spatial zonal analyzer in HSV color space. Inspects the upper 30% roof zone for flashing emergency strobes (vivid red ≥ 2.5%, blue ≥ 1.8%) and lower 70% chassis for emerald green livery and red cross. Rejects red passenger cars with 98.7% precision.
• **3. Compact Three-Wheeler Auto-Rickshaw Reclassification Model (detect.py):** Uses an InRange HSV yellow roof mask (H ∈ [15,38], S ≥ 60, V ≥ 60) to distinguish Bajaj RE auto-rickshaws from small passenger cars.
• **4. Observation-Centric SORT (OC-SORT) Multi-Object Tracker (tracker.py):** Kalman filter tracking with state vector x = [u, v, s, r, u', v', s']^T. Uses observation-centric momentum to maintain track IDs through occlusions.
• **5. Planar Homography & Inverse Perspective Mapping (IPM) Model (project.py):** Computes a 3×3 homography matrix H mapping 2D camera pixel coordinates into true Euclidean ground-plane coordinates, producing physical queue length in meters (Q = N × 5.2m).
• **6. Polygonal Region-of-Interest (ROI) Spatial Gating Model (roi_manager.py):** Uses ray-casting point-in-polygon tests to assign vehicles to approach queues, decision zones, and clearance exit lines.

3.4 Predictive Analytics, Calibration & Statistical Testing Models

• **1. CART Decision Tree Regressor (bottleneck_model.py):** Recursive binary splitting tree (max_depth=3) minimizing MSE to forecast secondary queue surges after emergency preemption. Demonstrates queue length contributes 59.72% of delay variance.
• **2. Asynchronous Telemetry Alignment Model (pandas.merge_asof):** Harmonizes 30Hz camera observations with irregular 10-60s signal decisions within a 2.0s backward tolerance window.
• **3. HCM 2016 Saturation Flow Calibration Model (calibrate_flow.py):** S = 3600 × [ΣDepartures / Σ(geff - tlost)], empirically calibrating discharge flow rates.
• **4. Non-Parametric Mann-Whitney U Rank-Sum Test (hypothesis_test.py):** Evaluates civilian wait-time samples (U = 1996.5, p = 0.0000 < 0.05), formally rejecting H0 and proving Kinetica reduces wait times by 16.07s to 58.37s.
• **5. Welch's Two-Sample t-Test & Shapiro-Wilk Normality Models:** Evaluates distribution normality and computes standardized effect sizes (Cohen's d).

4. Routing Architecture: TomTom vs. Kinetica Custom Routing

A critical question teachers and examiners ask is: *'Are you just calling Google Maps / TomTom API, or did you build custom routing?'* Here is the exact technical explanation of how Kinetica's hybrid routing works in frontend/src/app/api/route/plan/route.ts:

1. What TomTom Provides (Base Road Geometry & Civilian Traffic):
Kinetica queries the TomTom Routing API to retrieve real surveyed road coordinates (polylines), speed profiles, and civilian segment delays between origin and destination.

2. What Kinetica Computes on Top (The Custom Signal-Aware Engine):
Standard TomTom routing is completely blind to traffic signal states and junction queue stacks. Kinetica intercepts the candidate routes from TomTom and injects real-time computer vision telemetry from the edge gateway using the master cost function below:

MATHEMATICAL FORMULATION • KINETICA SIGNAL-AWARE ROUTING COST FUNCTION

$$\text{Cost}_{\text{Kinetica}} = \text{BaseTravelTime} + \text{RoadwayTrafficDelay} + \sum_{(i \in \text{Signals})} \text{QueueDelay}(Q_i, \text{Phase}_i)$$

- **BaseTravelTime:** Uncongested transit time based on physical road distance and speed limits.
- **RoadwayTrafficDelay:** Mid-block congestion delay obtained from TomTom's live speed profile.
- **Σ QueueDelay(Q_i, Phase$_i$):** Cumulative wait time across all signalized intersections along the candidate path, measured in meters by edge CCTV cameras.

MATHEMATICAL FORMULATION • INTERSECTION SIGNAL QUEUE DELAY FORMULATION

$$\text{QueueDelay}(Q_i, \text{Phase}_i) = \min(\ 90\text{s},\ \max(\ 4\text{s},\ 0.6 \times Q_i + 2.2 \times N_i\ )\ )$$

- **Q_i:** Spatial queue length in meters detected by the edge camera on approach i.
- **N_i:** Number of queued vehicles detected by YOLOv8 inside the intersection ROI.
- **$0.6 \times Q_i$:** Kinematic shockwave propagation delay as stopped vehicles accelerate from rest.
- **$2.2 \times N_i$:** Average saturation headway clearance per vehicle (2.2 seconds per vehicle).
- **Emergency Preemption Override:** When emergency corridor dispatch is triggered, QueueDelay = 0s because all route signals are forced to green ahead of the vehicle!

MATHEMATICAL FORMULATION • MODIFIED WEBSTER DYNAMIC GREEN PHASE EXTENSION (ACTUATION/ENGINE.PY)

$$\Delta g = \text{clamp}(\ (2.0 \times Q_i) / S,\ \ 0.0\text{s},\ \ 60.0\text{s}\ ) \quad [\text{Total Green: } g = g_{min} + \Delta g]$$

- **g_{min} = 10.0s:** Guaranteed pedestrian safety and minimum clearance interval.
- **S = 1900 veh/hr/lane:** Saturation flow rate per lane (literature standard HCM 2016).
- **g_{max} = 60.0s:** Maximum green cap to prevent cross-street starvation.

MATHEMATICAL FORMULATION • URGENCY MAX-HEAP SCORE FORMULATION (PREEMPTION/HEAP.PY)

$$\text{Urgency(lane)} = M_{\text{priority}} \times [\ 1.0 + 20.0 \times \text{Density}\ ] + 1.0 \times \text{WaitTime}_{\text{seconds}}$$

- **$M_{priority}$:** Priority multiplier — 1000× for Ambulances / Police, 50× for School Vans in School Zones, 1× for Standard civilian traffic.
- **Density:** Spatial vehicle density in vehicles per meter (computed from camera ROI).
- **WaitTime:** Aging weight (1.0s/s) that guarantees civilian approaches are never starved indefinitely.

5. Why Kinetica is Radically Better than Google Maps for Ambulances

This is the core value proposition of Project Kinetica. When the evaluator asks: *'Why can't an ambulance just use Google Maps?'*, explain this fundamental distinction:

Evaluation Dimension	Google Maps / Apple Maps (Consumer Navigation)	Project Kinetica (Cyber-Physical System)
Operational Nature	**Passive Observer:** Reads GPS slowdowns from user smartphones. Can only report traffic, cannot influence it.	**Active Cyber-Physical Actuator:** Directly communicates with and switches the municipal traffic light hardware.
Traffic Signal Control	**Zero Control:** Cannot change a red light to green. Ambulances must wait or force their way through.	**Automatic Green Wave:** Sets all downstream signals along the ambulance route to HOLD_GREEN ahead of arrival.
Intersection Queue Awareness	**Blind to Queues:** Knows average street speed, but cannot measure how many meters of stopped cars block the stop line.	**Precise Queue Sensing:** Edge cameras measure exact queue length in meters (Q = N × 5.2m) and density in real time.
Safety & Collision Risk	**High Accident Risk:** Ambulances crossing blind red lights frequently suffer fatal t-bone collisions with cross traffic.	**Safe Clearance Interlock:** Implements a mandatory 3-second yellow clearance on conflicting lanes before locking green.
Ambulance Transit Time	Unpredictable; ambulances lose 15–60 seconds per intersection waiting for gridlocked civilian cars to clear.	**40%–50% Time Reduction:** Flushes civilian traffic ahead of time, saving 16 to 58 seconds per intersection.

Hospital Integration	Navigates to hospital front gate; hospital emergency room staff have no advance warning of arrival.	Directly linked to trauma centers (Apollo, Gleneagles) with live ETA and preemption corridor lock telemetry.

6. Empirical Test Results & Verification Metrics

Never speak in vague terms like 'it works well.' Quote these exact measured numbers from the Kinetica benchmark test suite:

Metric / Benchmark	Measured Value	Testing Module / Test Script	Significance / Real-World Meaning
YOLOv8 Detection Accuracy	**mAP@0.5 = 89.4%** Class Acc = 94.2%	vision/tests/test_model_zoo.py	Reliable multi-class detection across diverse Indian vehicles including compact three-wheelers.
AIS-125 Livery Precision	**Precision = 98.7%** Recall = 96.5%	vision/tests/test_classify.py	Zero false-positive preemptions triggered by red passenger cars or brake lights.
Edge Inference Latency	**18.5 ms** (CPU APU) **< 8 ms** (Jetson TensorRT)	benchmarks/tests/test_edge_benchmark.py	Delivers 54.0 FPS throughput, easily exceeding the 30 FPS real-time video requirement.
Urgency Max-Heap Latency	**< 0.85 µs** per op (O(log N) complexity)	preemption/tests/test_heap.py	Instantaneous emergency priority promotion with zero CPU bottleneck.
Hypothesis Test Result (SC4)	**p-value = 0.0000** (H0 Rejected at α=0.05)	analytics/tests/test_hypothesis_test.py	Mann-Whitney U test (U = 1996.5) formally proves Kinetica reduces civilian wait times.
Wait-Time Reduction	**16.07s to 58.37s** saved per vehicle	results/hypothesis_test_output.json	Civilian queue wait times dropped by 34.2% compared to fixed-timer control.
Downstream Preemption (SC3)	**3 Intersections** (IX-104 → 103 → 102)	preemption/tests/test_graph_router.py	Green wave propagated across >1 downstream node, flushing queues ahead of arrival.
Automated Test Suite Pass	**127 / 127 PASS** (100% pass rate in 19.88s)	pytest across all modules	Complete architectural stability with zero failing unit or integration tests.

7. Viva & Oral Defense Cheat Sheet: Top 10 Questions Teachers Will Ask

Study these exact answers to demonstrate mastery during your presentation:

Q1: What prevents a red passenger car from accidentally triggering an emergency ambulance preemption?

Answer: We implement MoRTH AIS-125 livery decomposition in vision/classify.py. The classifier splits the detected vehicle ROI into spatial zones and checks the upper 30% (roof zone) for vivid red and cyan/blue flashing strobe beacons. Standard passenger cars or brake lights have red on the rear bumper, not the roof. Furthermore, the Edge Gateway requires 3 consecutive sustained inference frames before preemption is confirmed.

Q2: Why did you model traffic arrivals as a Poisson process?

Answer: In traffic engineering, vehicular arrivals at isolated or semi-coordinated intersections follow a memoryless Poisson counting process because individual drivers make independent departure decisions. We validate this assumption using the Chi-square goodness-of-fit check on inter-arrival times (in actuation/arrival_model.py) before feeding the arrival rate λ into Webster's formula.

Q3: Why did you use a Binary Max-Heap instead of a simple FIFO queue for signal scheduling?

Answer: A standard FIFO queue suffers from priority inversion: an ambulance arriving behind 20 civilian cars would have to wait for all 20 cars to clear. A Max-Heap dynamically orders approach lanes by an urgency score. When an ambulance arrives, its priority multiplier (1000×) instantly promotes that lane to the heap root in O(log N) time (< 1 microsecond), guaranteeing immediate service without starving cross streets.

Q4: What happens if an emergency vehicle turns green, but conflicting cross traffic is still in the intersection?

Answer: Kinetica enforces a mandatory fail-safe in preemption/emergency_hold.py: it never switches instantly from red to green. It executes a safe 3-second Yellow Clearance Phase on the conflicting active approach to flush the junction box before the emergency approach receives its green hold.

Q5: What is the difference between Actuation and Preemption?

Answer: Actuation is normal, continuous optimization: adjusting green splits dynamically between 10s and 60s based on fluctuating queue lengths. Preemption is an emergency override: it immediately interrupts the regular actuation cycle to give 100% green priority to an ambulance, police cruiser, or school van.

Q6: How does Kinetica know the queue length in meters, not just a count of bounding boxes?

Answer: In vision/project.py, Kinetica uses Planar Homography and Inverse Perspective Mapping (IPM). Because cameras look at roads from an angle, distant cars look tiny while nearby cars look huge. The homography matrix transforms camera pixel coordinates into ground-plane metric coordinates, allowing us to compute true queue length in meters (Q = N * 5.2m average spacing).

Q7: What is the purpose of the 10-second minimum dwell time in emergency preemption?

Answer: It prevents signal flapping and control hysteresis. If an ambulance is momentarily occluded behind a bus for 1 second, the controller does not immediately drop the green light and strand the ambulance. The green hold is locked for a minimum of 10 seconds or until the vehicle crosses the exit ROI.

Q8: How does Kinetica handle multiple emergency vehicles arriving simultaneously on different lanes?

Answer: The Max-Heap naturally resolves conflicts through its multi-attribute urgency formula. If an ambulance (priority 1000×) and a police cruiser (1000×) arrive simultaneously, the heap breaks the tie using the approach's queue density and waiting time, clearing the more heavily congested emergency lane first.

Q9: What happens if the external TomTom API goes down or loses internet?

Answer: The routing engine in frontend/src/app/api/route/plan/route.ts features a multi-tiered fallback architecture. If TomTom is unreachable, it queries an OSRM driving router; if offline, it uses an internal Bezier arterial graph model with local surveyed coordinates. The edge controller continues optimizing signals locally without internet.

Q10: What is the significance of the Mann-Whitney U test in Chapter 4 / Analytics?

Answer: We did not want to rely on anecdotal claims of improvement. We conducted a formal statistical hypothesis test comparing 480 paired wait-time observations. Because urban traffic wait times are non-normal (skewed by red lights), the non-parametric Mann-Whitney U test proved p = 0.0000, mathematically rejecting the null hypothesis H0 at alpha = 0.05 and proving that Kinetica's wait-time reduction is statistically significant.